

Инфраструктура как код (IaC) и CI/CD

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Илья Ящук

Data Scientist

- 5 лет опыта в Reliability Engineering, Condition Monitoring
- Кандидат технических наук (предиктивная аналитика)
- Анализ данных и разработка ML-моделей
- MLOps в RenMoney (FinTech)



@ilia_iashchuk



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

IaC

CI/CD

Практика

Заключение

Цели вебинара

К концу занятия вы сможете

Автоматизировать развёртывание и управление инфраструктурой с помощью инструментов IaC (например, Terraform);

Проектировать и выстраивать CI/CD-пайплайны для автоматической сборки, тестирования и доставки AI-приложений.

Смысл

Для чего вам это уметь

Управлять инфраструктурой и доставкой кода современными и удобными способами



Infrastructure as Code

Зачем IaC AI-архитектору?



Проблема без IaC

1. **Инфраструктура создаётся вручную** - воспроизвести сложно
2. **Нет версионирования:** непонятно, что и когда изменилось
3. **Для AI это критично:** эксперимент должен быть воспроизводим вместе с инфраструктурой



Что дает IaC

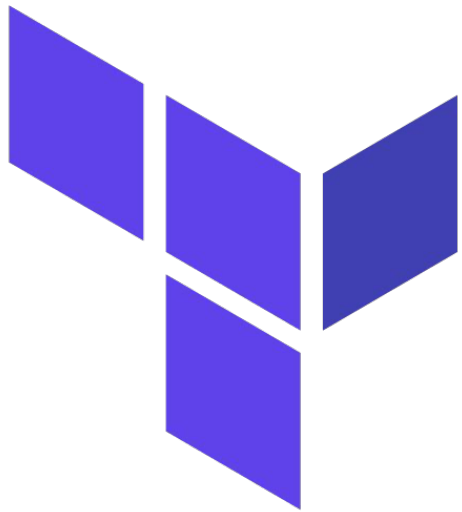
1. **Инфраструктура в Git** - как код, с ревью и историей
2. **Один конфиг разворачивает окружение:** dev / staging / prod
3. **Откат изменений так же прост** - используем Git команды



Инструменты IaC



Terraform



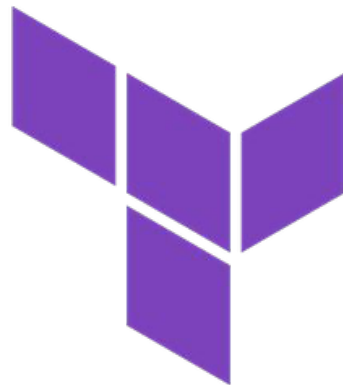
Terraform

Terraform — это инструмент для инфраструктуры как кода (Infrastructure as Code, IaC), который позволяет вам определять и управлять ресурсами инфраструктуры с помощью декларативных конфигурационных файлов.

Разработан Hashicorp

Написан на Go(lang)

Работает с облачными провайдерами



HashiCorp

Terraform

terraform.io



Как работает Terraform



Ключевые концепции Terraform

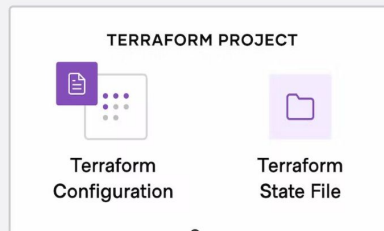
1. **Provider** - драйвер к облаку. Для YC: **yandex-cloud/yandex**
2. **Resource** - любой объект инфраструктуры: сеть, кластер, бакет
3. **State** - файл, в котором Terraform хранит что уже создано и чем управляет



Цикл работы Terraform

Write

Define infrastructure in configuration files



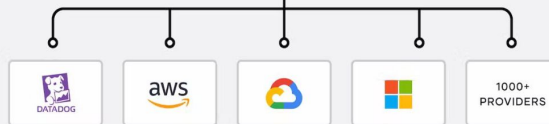
Plan

Review the changes
Terraform will make to
your infrastructure

```
$ terraform plan
...
Terraform will perform
the following actions
```

Apply

Terraform provisions
your infrastructure and
updates the state file.



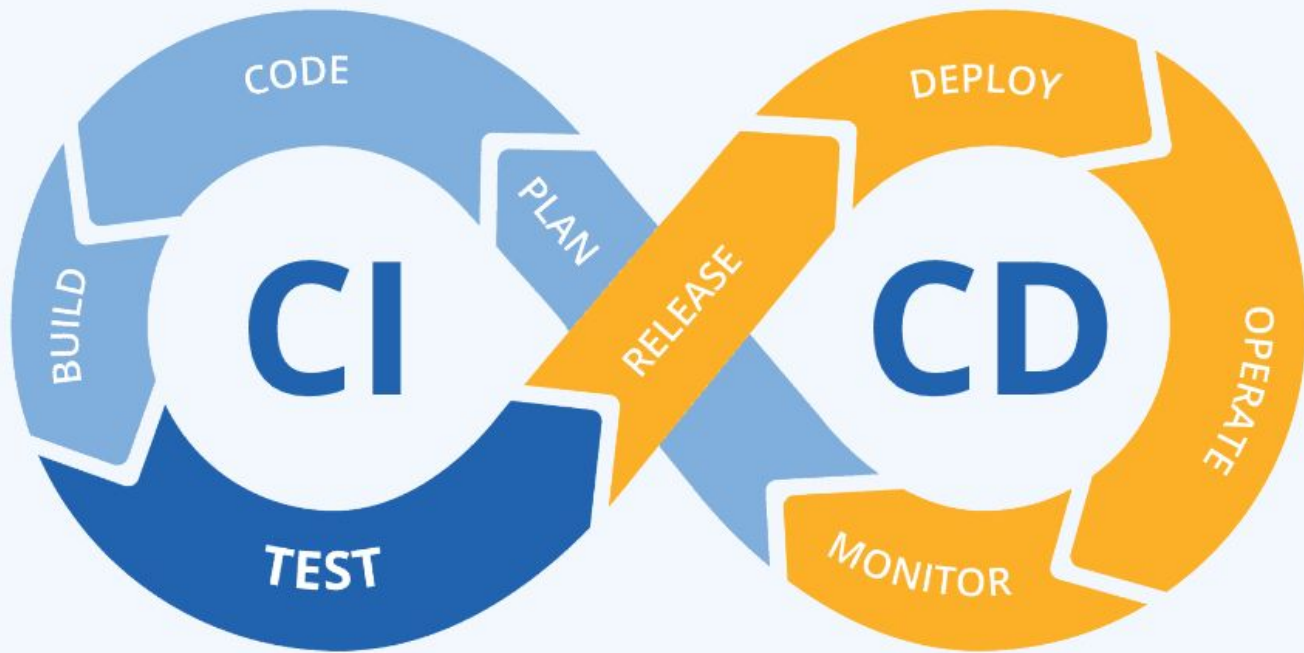
Цикл работы Terraform

```
terraform init      # скачать провайдер  
terraform plan      # показать что изменится  
terraform apply     # применить изменения  
terraform destroy   # удалить всё
```

Типичная инфраструктура в AI конфигах

Компонент	Зачем в AI-системе
VPC + подсети	изоляция окружений, безопасность, зоны доступности
Managed Kubernetes	запуск контейнеров: inference, training, preprocessing
Object Storage	хранение датасетов, чекпоинтов, артефактов моделей
Container Registry	версионирование Docker-образов сервисов
Managed PostgreSQL	хранение метаданных экспериментов, feature store
IAM + сервисные аккаунты	разграничение прав между компонентами

CI/CD



Отличие CI/CD для AI от классического Software

Этап	Software	AI
Артефакт	Docker-образ	образ + веса модели + конфиг
Тестирование	unit / integration	unit / integration + качество модели, метрики
Откат	предыдущий образ	предыдущий образ + предыдущая версия модели
Триггер	push в репозиторий	push + новые данные, дрейф модели

Главная сложность

модель может сломаться не из-за бага в коде, а из-за изменения данных или гиперпараметров.

Этапы пайплайна



Пайплайн CI/CD например в GitLab

Этапы	Описание
START	push в GitLab - старт пайплайна
BUILD	сборка образа
	push образа в CR
TEST	unit-тесты
	интеграционные тесты API
SECURITY	проверки безопасности - SAST, проверка контейнеров
VALIDATE	загрузка весов из S3
	прогон на validation dataset
	проверка метрик
DEPLOY	обновление Helm чарта в k8s
MONITORING	реальный запрос к inference
	проверка ответа и времени отклика

Quality Gates

Gate	Инструменты	Критерий остановки
Линтеры	ruff, black, isort	любая ошибка
Линтеры Terraform, k8s	tf validate, tflint, helm lint	любая ошибка
Unit-тесты	pytest	coverage < порога %
Безопасность	semgrep, trivy	критические уязвимости, секреты
Качество модели	custom script	метрика < порога
Smoke-тесты	curl	статусы, latency



Практика

Цель этого этапа: дать студентам задание для самостоятельной/групповой работы. Можно попросить студентов решить тест/кейс/задачу

Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

Теперь вы сможете:

Автоматизировать развёртывание и управление инфраструктурой с помощью инструментов IaC (например, Terraform);

Проектировать и выстраивать CI/CD-пайплайны для автоматической сборки, тестирования и доставки AI-приложений.

Управлять инфраструктурой и доставкой кода современными и удобными способами

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

